

Section 1.3 — Working Notes

Study of the Existing System and Critical Analysis

Notes only. Write the sentences yourself.

Before you start

- **Write ugly first.** Get every point down as a bad sentence. Fix later. The blank page is the expensive part, not the editing.
- **One idea per paragraph.** If it has two, split it.
- **Say the point, then the evidence.** Not the other way round. Building to a conclusion is harder to read and harder to write.
- **Close this file before you write.** With the notes open you copy phrasing instead of thinking.
- **Every claim about your own system must be one you can defend out loud.** If you would not say it to the jury, do not write it.

Order to write in (not the order they appear):

1.3.1.3 → 1.3.1.1 → 1.3.2.2 → 1.3.2.1 → 1.3.2.3 → 1.3.1.2 → 1.3.2.4 → bridges last.

Start with 1.3.1.3: most concrete, you know it cold, and it sets the tone for everything else. Bridges are easy once you know what you are bridging.

1 1.3 — Opening bridge

Job: Close off 1.2, announce the two halves. Two or three sentences, no more.

- 1.3.1 establishes what the document demands of anyone consulting it
- 1.3.2 measures existing tools against those demands
- why this order: you cannot judge a tool before you know the job

Point to land: *Nothing in section 1.4 is invented. It is all derived from what follows here.*

2 1.3.1 — Bridge

Job: One or two sentences. Do not stack headings.

- three things coming: what the matrix is / what rides on reading it correctly / what makes reading it hard

3 1.3.1.1 — Definition and Stakes of the DAM

3–4 paragraphs.

Para 1 — what delegation is

- authority formally concentrated at the top: Board, Board of Governors, President
- cannot operate that way — thousands of decisions, one institution
- so authority is delegated downward, but in a controlled and *recorded* way
- the DAM is that record
- not a procedures manual — it never says how to do a thing

Point to land: *The DAM answers “who may”. It never answers “how”.*

Para 2 — the shape of the document

- a matrix, two axes
- **rows = activities**, hierarchical, four levels: chapter → process → task → child task
- plus *threshold variants*: same task, different money band, different approver
- the hierarchy is structural, not textual — 2.513 sits under process 2.510, but the identifier string does not say so. You have to know the rule.

- **columns = roles, not people**
- why that matters: the document survives staff turnover; also why your system resolves roles and not names
- **cell = an authority code**

Para 3 — the codes, and two traps

- point the reader at your codes table, do not re-list it in prose
- **Trap 1 — “C” is overloaded.** C1/C2 = check and verify, an operational control step. C3/C4 = consult, typically Board, Board of Governors, or an independent function. Same letter, unrelated obligations, different organisational level. A system that maps the word “check” onto a bare C merges them and returns something wrong that looks right.
- **Trap 2 — the informed marker is not a letter.** It is parenthesised, (i). So it behaves differently at every stage: extraction, storage, retrieval, phrasing. Tiny notation difference, disproportionate engineering cost.
- worth saying: these are not exotic edge cases. They are on nearly every page.

4 1.3.1.2 — Governance and Compliance Requirements

3–4 paragraphs. This is the “why it matters” subsection — do not let it drift abstract.

Para 1 — the status of the document

- not advisory, not guidance — it is the control baseline
- internal audit tests against it
- exists partly because public and multilateral funding brings governance expectations from member states
- therefore a wrong reading is not a mistake, it is a control breach

Para 2 — consequence 1: the failure is delayed

- nothing breaks on the day a wrong approval is given
- the payment goes out, the contract is signed, the project proceeds
- it surfaces at audit — months later, sometimes longer
- contrast with software errors, which announce themselves: an exception, a 500, a red test
- this one is silent at the moment it is made
- design consequence: you cannot rely on the user noticing a wrong answer

Para 3 — consequence 2: completeness is part of correctness

Biggest idea in the subsection. Give it its own paragraph and do not rush it.

- an answer that names the approver but omits a mandatory verifier contains nothing false
- it still misleads — the user acts on it and skips a required control
- so an incomplete true answer can do the damage of a false one
- say explicitly that FR4 comes from here
- flag that this is unusual: in most question-answering systems a partial answer is acceptable. Here it is not. That asymmetry is worth naming.

Para 4 — consequence 3: traceability

- an answer the user cannot check gives no rational basis for trust
- without it you replace one opaque authority — a PDF nobody reads — with a worse one: a model nobody can audit
- design consequence: every answer carries its source activity
- second reason, worth a clause: traceability is what makes the system defensible to an auditor, not merely useful to a user

5 1.3.1.3 — Challenges in Consulting the DAM in Practice

4 paragraphs. Write this one first. Be concrete — the jury has never opened this document.

Para 1 — the geometry

- tables are very wide: a dozen or more role columns on a fixed page width
- headers rotated ninety degrees, read bottom to top
- readable, but effortful — you tilt your head, or rotate the screen
- **the failure mode is adjacency** — land this hard:
 - neighbouring roles often hold different authority over the same activity
 - one column out is not slightly wrong, it is completely wrong
 - nothing on the page tells you it happened
 - a printed table has no error message

Para 2 — the footnotes

- superscript digits sit jammed against the codes
- the footnotes carry the conditions: ceilings, circumstances, exceptions
- the reader separates three things by eye — the letter, the level digit, the footnote digit
- an “A” with a small 3 beside it: is that code A with footnote 3, or something else?
- **your measured number: 113 of 1,776 responsibility entries carry a footnote reference — 6.36%**
- that number is useful precisely because it looks ignorable and is not: the 6.36% are disproportionately the conditional, high-value cases
- your extraction resolves this with character offset and font size — mention as a forward reference only, the detail belongs in Chapter 2

Para 3 — the redirects

- some activities record no responsibilities at all
- instead, a pointer: “See DAM 16.100”, “Refer to Activities 2.114–2.117 in Section 2”
- you navigate to the thing you wanted and are told to start again elsewhere
- for a human: annoying
- for a naive system: it reports “no responsibilities found”, which is *wrong*, not merely unhelpful
- FR5 comes from here

Para 4 — the behavioural point

Strongest paragraph in 1.3.1. Save it for last and say it plainly.

- because consultation is expensive, people avoid it
- they ask a colleague who did it last time
- or they repeat what was done before without rechecking
- neither is auditable
- and an error propagates indefinitely — once “we have always done it this way” sets in, nobody reopens the page
- so the real cost of the document’s format is not slow lookups. It is that lookups stop happening.

Point to land: *That is the problem the project actually addresses. Everything else is a symptom.*

Figure — the most important one in the chapter

- schematic mock-up of a DAM page. Invent the activity and role names — no confidentiality issue that way, and it reads more clearly than a redacted screenshot
- must show: wide table, rotated headers, codes in cells, one superscript, one redirect row
- add a callout on a single cell: “A³” — is the 3 part of the code, or a footnote marker?
- that one callout is the entire geometry problem in a single image

6 1.3.2 — Bridge

- four categories of tool could be aimed at this
- none works

- each fails for a different, specific reason
- those reasons are what become the requirements

Point to land: *This is not a survey. It is where the specification comes from.*

7 1.3.2.1 — Manual Consultation and Document Search

2 short paragraphs.

Para 1 — what people do now

- open the PDF, Ctrl+F, that is the whole workflow
- full-text search is character string matching — no model of the document, no table awareness, no relevance ranking
- works only if you already know the exact printed title or the identifier
- “approve” returns hundreds of hits in page order
- a half-remembered activity name returns nothing

Para 2 — the deeper failure

- search returns a *location*. The user needs an *answer*.
- it tells you the phrase is on page 24
- it cannot tell you who approves, because that requires knowing the page holds a table, that headers are rotated, and that a character in a cell denotes a role’s authority
- none of that exists in a string match

Point to land: *This is a difference of category, not a gap in quality. A better search engine does not close it.*

8 1.3.2.2 — Generic Document Question-Answering Assistants

3 paragraphs.

Para 1 — what they are

- named examples: ChatPDF, NotebookLM, Microsoft Copilot, the file-upload modes of the major chat assistants
- state the mechanism accurately, you will be asked: document split into passages → each embedded as a vector → passages nearest the question retrieved → language model writes from those
- this is RAG. Use the term — it will be expected.

Para 2 — why it fails here

- works well on prose
- fails on the DAM, and the failure is **structural, not a matter of model quality**
- this matters: if a reader leaves thinking a better model would fix it, you have lost the argument

Para 3 — the chain of failure

- step one flattens the page to text
- but this page’s meaning is *positional* — an “A” sitting under a particular rotated header *is* the content of the cell
- flattening keeps the characters and discards the geometry
- the rotated header becomes a loose string attached to nothing
- so a retrieved passage can contain the activity name *and* the letter A, having lost which column the A was in
- the answer is destroyed before the model is ever involved
- and the model writes something fluent from what is left

Point to land: *That last part is what makes these tools dangerous rather than merely useless.*

Figure — 15 minutes, highest value you have

- upload two or three DAM pages to ChatPDF or NotebookLM
- ask: “who approves 2.126”
- screenshot whatever comes back
- if it is wrong or vague, that screenshot turns this subsection from an argument into evidence — and it is the kind of thing a jury remembers

9 1.3.2.3 — Commercial Document-Extraction Services

2 paragraphs.

Para 1 — be fair to them first

- different category — they do not answer questions, they extract structure
- named: Azure Document Intelligence, AWS Textract, Google Document AI, and layout-aware parsers such as LlamaParse
- genuinely good at tables, markedly better than a general-purpose library
- these are the *serious* alternative to hand-written rules. A jury will ask why you did not just use one, so answer it head-on. Conceding their strength makes you more credible, not less.

Para 2 — three reasons ruled out

- **1. Cost and confidentiality.** Priced per page, and it means sending an internal governance document to a third party. That is an institutional decision, not a student’s. Note honestly that this alone would not kill them.
- **2. Fit.** Pre-trained for invoices, receipts, IDs, tax forms, contracts. A bespoke authority matrix is not a document type anyone pre-trained for, so you fall back to generic table extraction anyway.
- **3. Explainability — the one that actually decided it. Give it the most room.**
 - a wrong cell gives you a lower confidence score and nothing else
 - you cannot ask why, cannot trace the decision, cannot fix it locally
 - compare: your superscript rule came from measured characters on a real page, so when it misfires you open that page and see exactly why
 - in a document where a wrong answer looks identical to a right one, interrogable extraction beats unexplainable accuracy
- note the through-line: this is the same reason you verify the model’s output rather than trusting the prompt. Saying so makes the thesis read as one argument instead of five chapters.

10 1.3.2.4 — Critical Analysis: Limitations of Current Approaches

3 paragraphs, then the comparison table.

Para 1 — refuse the easy conclusion

- none of them fail from poor engineering
- each fails on an *assumption* that does not hold here:
 - search assumes the user can phrase it in the document’s own vocabulary
 - assistants assume flattening preserves meaning — false for positional tables
 - extraction services assume the document resembles their training forms
 - GRC platforms (the fourth category) assume the institution will re-enter its rules into their data model — that is a change programme, not a tool

Para 2 — the one blind spot they share

Biggest idea in 1.3.2. Own paragraph. Put weight here.

- none of them has any concept of being wrong
- search returns results; a model returns text
- neither separates “an answer I can support” from “one I cannot”

- with no internal notion of “I do not know”, they never say it
- for a compliance reference, that distinction is the entire product

Point to land: *That is the gap this system is built in.*

Para 3 — hand off

- point at the comparison table
- closing move: read down your own column. That list *is* your specification. Say it, then point forward to 1.5.

Table caution

- do not mark a competitor “No” on anything you have not tested
 - “N/A” and “Partly” are defensible under questioning. A confident “No” is not.
-

Your numbers, for cross-reference

- DAM: 250+ pages total, 79-page subset processed, three chapters
- 327 records, 1,776 responsibility entries, 131 roles
- 113 of 1,776 entries carry a footnote reference (6.36%)
- 0.56% unresolved role rate
- graph: 459 nodes, 2,108 edges
- 374 tests across 23 files